VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Logic Design with HDL (CO1026)

Assignment Report

Designing a simple RISC Processor

Instructor: Nguyễn Thành Lộc

Students:	Trần Vinh Quang	2550342
	Lê Hiển Vinh	2453422
	Nguyễn Đức Thiên Tân	2550345
	Đặng Ngọc Lan Anh	2550258

HO CHI MINH CITY, JUNE 2026

Contents

1 Introduction **4**
1.1 Project Overview 4
1.2 Objectives 4
1.3 Tools and Environment 5
1.4 Main Features of the CPU 5
1.5 Report Organization 6

2 System Design Approach **6**
2.1 Theoretical Overview 6
2.2 Top-level System Architecture 8
2.3 Fetch Phase 8
2.4 Execution Phase 8
2.5 Functional Blocks 9

3 Implementation **13**
3.1 Implementation Strategy 13
3.2 Program Counter (PC) 13
3.3 Address Multiplexer (Address MUX) 14
3.4 Arithmetic Logic Unit (ALU) 15
3.5 Controller Unit 17
3.6 Instruction Register (IR) 22
3.7 Accumulator Register (AC) 23
3.8 Memory 25
3.9 Top-level Integration 26

4 Testing Simulation and Validation **29**
4.1 Program Counter (PC) 29
4.2 Address Multiplexer (Address MUX) 33
4.3 Arithmetic Logic Unit (ALU) 37
4.4 Controller Unit 42
4.5 Instruction Register (IR) 48
4.6 Accumulator Register (AC) 52
4.7 Memory 56
4.8 Top-level Integration 59

5 Conclusion and Future Work **65**
5.1 Conclusion 65
5.2 Future Plan 66

1 Introduction

1.1 Project Overview

A Central Processing Unit (CPU) is the core component of a computer system, responsible for fetching instructions, decoding them, executing operations, and controlling data movement between internal registers and memory. In this project, a simple Reduced Instruction Set Computer (RISC) CPU is designed using *Verilog Hardware Description Language (HDL)*.

The proposed CPU follows a simplified RISC architecture with a compact instruction format. Each instruction consists of a 3-bit opcode and a 5-bit operand field, allowing the processor to support eight basic instructions and address a memory space of 32 locations. Although the architecture is simple, it demonstrates the fundamental principles of processor design, including instruction fetching, instruction decoding, arithmetic and logic execution, memory access, and program control.

The CPU is organized as a set of functional hardware modules, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, Arithmetic Logic Unit, and Controller. These modules are designed independently and then integrated into a complete processor system. The CPU operates based on clock and reset signals and continues executing instructions until a halt instruction is encountered.

1.2 Objectives

The main objective of this project is to design, implement, and verify a simple RISC CPU at the register-transfer level using Verilog HDL. Through this project, the design process of a basic processor can be studied from both theoretical and practical perspectives.

The specific objectives of the project are as follows:

- To understand the fundamental organization and operation of a RISC-based CPU.
- To design the main functional blocks of the CPU, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, ALU, and Controller.
- To implement each functional block using Verilog HDL with a modular and hierarchical design approach.
- To integrate all modules into a complete CPU datapath and control path.
- To verify the correctness of each module through individual testbenches.
- To evaluate the complete CPU by observing simulation waveforms and checking the execution of supported instructions.

By completing these objectives, the project provides practical experience in digital system design, hardware modeling, finite state machine design, and simulation-based verification.

1.3 Tools and Environment

The main tool used in this project is Vivado Design Suite. Vivado provides an integrated development environment for designing, simulating, and verifying digital hardware systems described using Verilog HDL. In this project, Vivado is used to create Verilog source files, develop testbenches, run simulations, and analyze signal waveforms. The source code is formatted, linted, and maintained using `Verible` and the SystemVerilog Language Server (`SVLS`) to ensure strict hardware coding conventions.

Each CPU component is implemented as an independent Verilog module and verified using a corresponding testbench. A custom `Python` script is implemented with Icarus Verilog (`iverilog`) serves as the primary compiler for rapid, iterative command-line simulations and unit testing. After unit-level verification, the modules are connected together in a top-level design to perform system-level simulation. The waveform viewer in Vivado is used to observe the behavior of internal signals, such as the program counter value, memory address, instruction register output, accumulator value, ALU result, and controller signals.

Vivado is suitable for this project because it supports hierarchical hardware design, Verilog-based modeling, testbench simulation, and detailed waveform analysis. These features allow the design to be verified systematically before further hardware synthesis or implementation.

1.4 Main Features of the CPU

The designed CPU supports a small but representative set of instructions. Since the opcode field is 3 bits wide, the CPU can define up to 8 instructions. These instructions include program control, arithmetic operations, logic operations, memory access, and conditional execution.

The main features of the CPU are summarized as follows:

- A 3-bit opcode field supporting eight basic instructions.
- A 5-bit operand field supporting 32 memory address locations.
- A Program Counter used to store and update the current instruction address.
- An Address Multiplexer used to select between the instruction address and operand address.
- A Memory module used to store both instructions and data.
- An Instruction Register used to hold and decode the current instruction.

- An Accumulator Register used to store intermediate and final computation results.
- An ALU used to perform arithmetic and logic operations.
- A Controller implemented as a finite state machine to generate control signals for instruction execution.
- A halt mechanism that stops program execution when the halt instruction is detected.

The CPU performs its operation through a sequence of basic stages: instruction fetch, instruction decode, operand access, execution, and result storage. This execution flow reflects the essential behavior of a real processor while remaining simple enough for analysis, implementation, and verification.

1.5 Report Organization

This report is organized into five main sections.

Section 1 introduces the project, including the project overview, objectives, tools and environment, main CPU features, and report structure.

Section 2 describes the theoretical background and the design of the CPU. This section presents the overall system architecture, block diagram, and detailed functions of each hardware module.

Section 3 discusses the implementation of the CPU using Verilog HDL. It explains the module structure, implementation approach, controller finite state machine, memory organization, and top-level integration.

Section 4 presents the testing and evaluation process. It includes unit-level testbenches, system-level simulation, waveform analysis, and functional evaluation of the CPU.

Section 5 concludes the report by summarizing the achieved results, discussing difficulties encountered during the project, and proposing possible future improvements.

2 System Design Approach

2.1 Theoretical Overview

Based on the project specification, the RISC CPU designed in this assignment features a 3-bit opcode and a 5-bit operand. This architecture supports 8 types of instructions (HLT, SKZ, ADD, AND, XOR, LDA, STO, JMP) and a 32-word memory address space. The processor operates synchronously with a clock signal and an active-high reset signal (the default reset state is `INST_ADDR`). Program execution stops upon encountering the `HLT` instruction.

By analyzing the controller state table and the functional requirements of each block, the CPU's operation flow can be divided into two primary phases spanning 8 clock cycles: the Fetch Phase and the Execution Phase.

Table 2.1: CPU Controller state and Control signals

Outputs	**Phase**								**Notes**
	`INST_ADDR`	`INST_FETCH`	`INST_LOAD`	`IDLE`	`OP_ADDR`	`OP_FETCH`	`ALU_OP`	`STORE`	
sel	1	1	1	1	0	0	0	0	ALU OP = 1 if opcode is ADD, AND, XOR or LDA
rd	0	1	1	1	0	ALUOP	ALUOP	ALUOP	
ld_ir	0	0	1	1	0	0	0	0	
halt	0	0	0	0	HALT	0	0	0	
inc_pc	0	0	0	0	1	0	SKZ && zero	0	
ld_ac	0	0	0	0	0	0	0	ALUOP	
ld_pc	0	0	0	0	0	0	JMP	JMP	
wr	0	0	0	0	0	0	0	STO	
data_e	0	0	0	0	0	0	STO	STO	

2.2 Top-level System Architecture

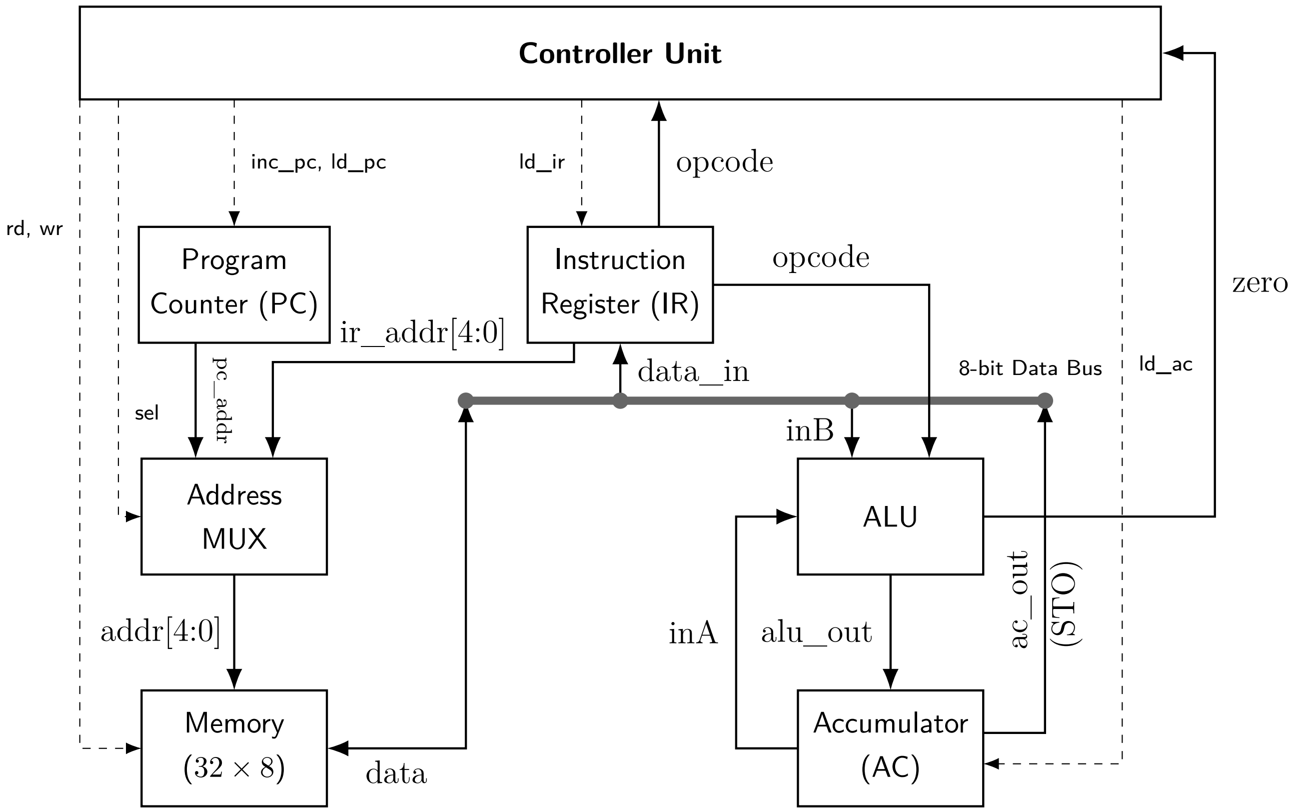


Figure 2.1: Top-Level Architecture of the RISC Processor

2.3 Fetch Phase

- **State 1: INST_ADDR.** The Program Counter (PC) provides the address of the current instruction. The Address Mux selects the PC address (`sel = 1`) to access the memory.
- **State 2: INST_FETCH.** The memory read signal (`rd = 1`) is asserted. The instruction data is retrieved from Memory.
- **State 3: INST_LOAD.** The fetched instruction is loaded into the Instruction Register (IR) via the `ld_ir = 1` control signal.
- **State 4: IDLE.** An intermediate wait state for the system to stabilize.

2.4 Execution Phase

- **State 5: OP_ADDR.** The IR splits the instruction into a 3-bit Opcode and a 5-bit Operand Address. The Address Mux switches to select the operand address from the IR (`sel`

= 0). Simultaneously, the PC increments (`inc_pc = 1`) to prepare for the next instruction cycle.

- **State 6: OP_FETCH.** If the instruction requires data from memory (e.g., ADD, AND, LDA), the CPU reads the data from the operand address in the Memory (`rd = 1`).
- **State 7: ALU_OP.** The Arithmetic Logic Unit (ALU) performs the required computation between the data fetched from Memory and the data currently held in the Accumulator (AC), based on the decoded Opcode. If the instruction is `SKZ` and the `zero` flag condition is met, the PC increments once more (`inc_pc = 1`) to skip the subsequent instruction.
- **State 8: STORE.** The result of the operation is either stored back into the Memory (for the `STO` instruction, asserting `wr = 1` and `data_e = 1`) or loaded into the Accumulator (for arithmetic and logic instructions, asserting `ld_ac = 1`).

2.5 Functional Blocks

Program Counter (PC)

- **Function:**
  - The counter is a crucial component used to track and count program instructions.
  - It can also be utilized to count program states.
  - The counter operates synchronously on the rising edge of the `clk` signal.
  - The `reset` signal is active-high, clearing the counter back to 0.
  - The counter has a width of 5 bits.
  - It features a load function to force an arbitrary address into the counter. If not loading, the counter increments normally.
- **Inputs:** `clk`, `rst`, `ld_pc`, `inc_pc`, `data_in`.
- **Outputs:** `pc_out`.

Address Multiplexer (Address MUX)

- **Function:**
  - The Address MUX selects between the instruction address (during the instruction fetch phase) and the operand address (during the instruction execution phase).
  - The MUX has a default data width of 5 bits.

  - The width is parameterized to allow for easy modification if necessary.

- **Inputs:** pc_addr, ir_addr, sel.
- **Outputs:** addr_out.

Arithmetic Logic Unit (ALU)

- **Function:**
  - The ALU executes arithmetic and logical operations. The specific operation performed depends on the instruction's opcode.
  - The ALU executes 8 distinct operations on 8-bit operands (inA and inB).
  - It produces a 8-bit output and a 1-bit asynchronous zero flag.
  - The zero flag indicates whether the input inA is equal to 0.
- **Inputs:** inA, inB, opcode.
- **Outputs:** alu_out, zero.

Table 2.2: ALU Opcode Lookup Table

Opcode	Code	Operation Description	Output
`HLT`	000	Halts program execution.	`inA`
`SKZ`	001	Checks if the ALU result is equal to 0. If zero, it skips the next instruction; otherwise, execution continues normally.	`inA`
`ADD`	010	Adds the value in the Accumulator to the memory value at the instruction's address, returning the result to the Accumulator.	`inA + inB`
`AND`	011	Performs a bitwise AND on the Accumulator value and the memory value at the instruction's address.	`inA AND inB`
`XOR`	100	Performs a bitwise XOR on the Accumulator value and the memory value at the instruction's address.	`inA XOR inB`
`LDA`	101	Reads the value from the address in the instruction and loads it into the Accumulator.	`inB`
`STO`	110	Writes the Accumulator's data to the memory address specified in the instruction.	`inA`
`JMP`	111	Unconditional jump; jumps to the target address in the instruction and continues execution.	`inA`

Controller Unit

- **Function:** The central finite state machine that decodes instructions and dictates the operation of all other blocks via control signals.
- **Inputs:** `clk`, `rst`, `opcode`, `zero`.
- **Outputs:** `sel`, `rd`, `ld_ir`, `halt`, `inc_pc`, `ld_ac`, `ld_pc`, `wr`, `data_e`.

Table 2.3: Control Execution Scenarios by Opcode

Instruction Scenario	Active States	Expected Control Signals
HLT (000)	OP_ADDR	halt = 1
ADD, AND, XOR, LDA	OP_FETCH to STORE	rd = 1, ld_ac = 1 (at STORE phase)
STO (110)	ALU_OP, STORE	wr = 1, data_e = 1
JMP (111)	ALU_OP, STORE	ld_pc = 1

Instruction Register (IR)

- **Function:** As described in the reference documentation, it captures and holds the current instruction fetched from memory, splitting it into an opcode and an operand.
- **Inputs:** clk, rst, ld_ir, data_in.
- **Outputs:** opcode, operand.

Accumulator Register (AC)

- **Function:** As described in the reference documentation, it acts as the primary working register for ALU operations and data transfers.
- **Inputs:** clk, rst, ld_ac, data_in.
- **Outputs:** ac_out.

Memory

- **Function:**
  - The memory module stores both instructions and data.
  - It is designed to separate read and write functions while utilizing a single bidirectional data port. Reading and writing cannot occur simultaneously.
  - Configured for 5-bit addresses and 32-bit data.
  - Uses 1-bit control signals to enable reading (rd) and writing (wr).
  - Memory operations are synchronized to the rising edge of the clk signal.
- **Inputs:** clk, rd, wr, addr.
- **Inout:** data.

3 Implementation

3.1 Implementation Strategy

The implementation follows a bottom-to-up design flow consistent with industrial RTL practice. The process went through 4 phases: specification review, RTL coding for each individual module, functional simulation, and unit testing for debugging to ensure any failures happened only in the current development module and that the hardware matched the intended behavior.

Throughout the implementation, the following coding style guidelines were enforced for readable, synthesizable, and maintainable Verilog source code: maintaining consistent indentation, naming conventions (`lower_case` variable casing), and comprehensive comments throughout the code.

3.2 Program Counter (PC)

Implementation Logic & Steps:

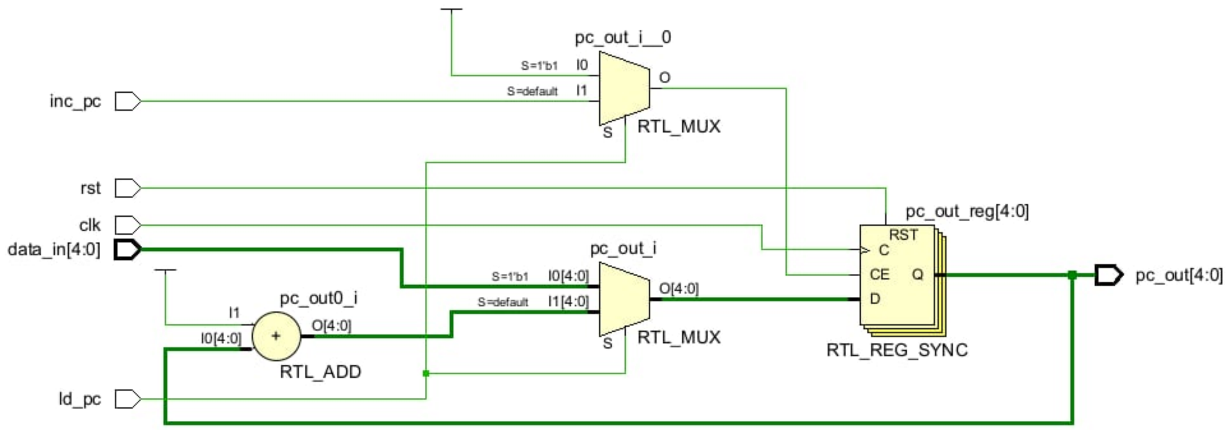
- Implemented a sequential logic block triggered by the positive edge of the clock.
- Implemented a synchronous active-high reset to clear the program counter.
- Implemented logic to load a new address into the program counter when `ld_pc` is active.
- Implemented logic to increment the program counter when `inc_pc` is active.

Source Code: `PC.v`:

```
module PC (
    input             clk,
    input             rst,
    input             ld_pc,
    input             inc_pc,
    input       [4:0] data_in,
    output reg  [4:0] pc_out
);

  always @(posedge clk) begin
    if (rst) begin
      pc_out <= 5'd0;
    end else if (ld_pc) begin
      pc_out <= data_in;
```

```
    end else if (inc_pc) begin
      pc_out <= pc_out + 5'd1;
    end else begin
      pc_out <= pc_out;
    end
  end

endmodule
```

RTL Schematic:



3.3 Address Multiplexer (Address MUX)

Implementation Logic & Steps:

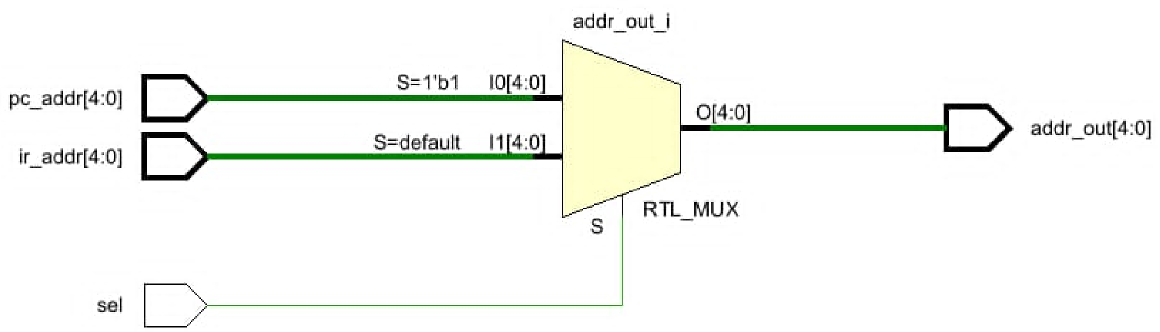
- Implemented combinational logic to select between `pc_addr` and `ir_addr` based on the `sel` signal.
- Routed `pc_addr` to the output when `sel` is 1, and `ir_addr` when `sel` is 0.

Source Code: `ADD_MUX.v`:

```
module address_mux #(
    parameter WIDTH = 5
) (
    input  [WIDTH-1:0] pc_addr,
```

```
    input  [WIDTH-1:0] ir_addr,
    input              sel,
    output [WIDTH-1:0] addr_out
);

  assign addr_out = (sel) ? pc_addr : ir_addr;

endmodule
```

RTL Schematic:



3.4 Arithmetic Logic Unit (ALU)

Implementation Logic & Steps:

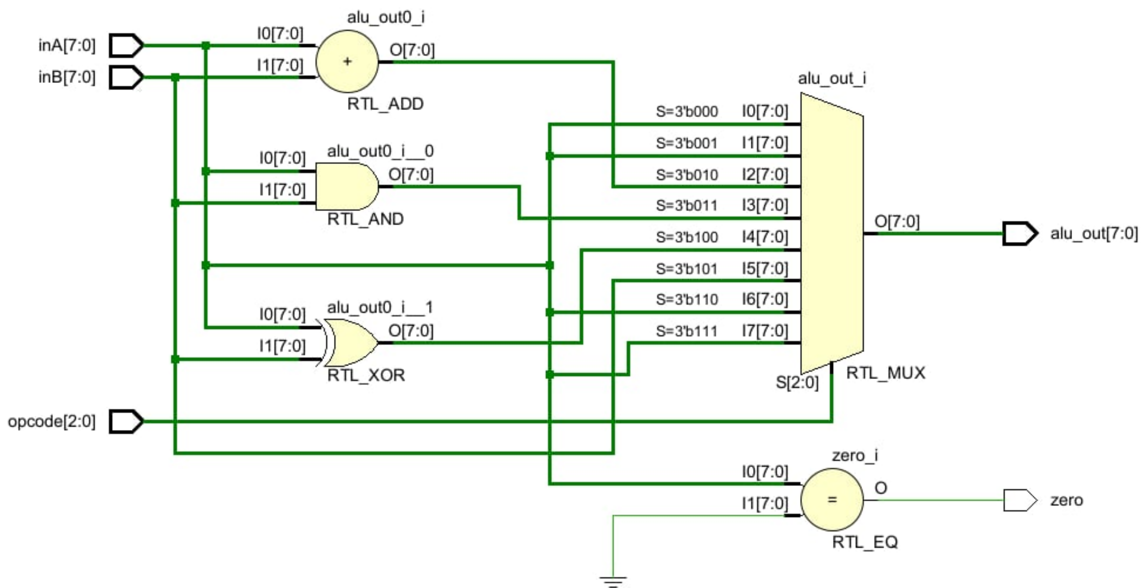
- Implemented combinational logic to set the zero flag if `inA` is 0.
- Implemented combinational logic to perform arithmetic and logic operations based on the opcode.
- Defined operations for HLT, SKZ, ADD, AND, XOR, LDA, STO, and JMP instructions.
- Assigned the combinational result to `alu_out`.

Source Code: `ALU.v`:

```
module ALU (
    input      [2:0] opcode,
    input      [7:0] inA,
    input      [7:0] inB,
    output reg [7:0] alu_out,
```

```
    output            zero
);
  always @(*) begin
    case (opcode)
      3'b000:  alu_out = inA;
      3'b001:  alu_out = inA;
      3'b010:  alu_out = inA + inB;
      3'b011:  alu_out = inA & inB;
      3'b100:  alu_out = inA ^ inB;
      3'b101:  alu_out = inB;
      3'b110:  alu_out = inA;
      3'b111:  alu_out = inA;
      default: alu_out = 8'b00000000;
    endcase
  end
  assign zero = (inA == 8'b00000000);

endmodule
```

RTL Schematic:



3.5 Controller Unit

Implementation Logic & Steps:

- Defined state parameters for the 8-state FSM.
- Defined opcode parameters for the 8 instructions.
- Declared a state register.
- Implemented a sequential logic block for FSM state transitions (triggered by positive clock edges).
- Implemented a synchronous active-high reset to set the initial state.
- Implemented combinational logic to determine output control signals based on the current state and opcode.
- Ensured all output signals were assigned a default value to prevent unwanted latches.

Source Code: `Controller.v`:

```
module controller (
    input            clk,
    input            rst,
    input      [2:0] opcode,
    input            zero,
    output reg       sel,
    output reg       rd,
    output reg       ld_ir,
    output reg       halt,
    output reg       inc_pc,
    output reg       ld_ac,
    output reg       ld_pc,
    output reg       wr,
    output reg       data_e

);
  localparam  INST_ADDR=3'd0,
              INST_FETCH=3'd1,
              INST_LOAD=3'd2,
              IDLE=3'd3,
```

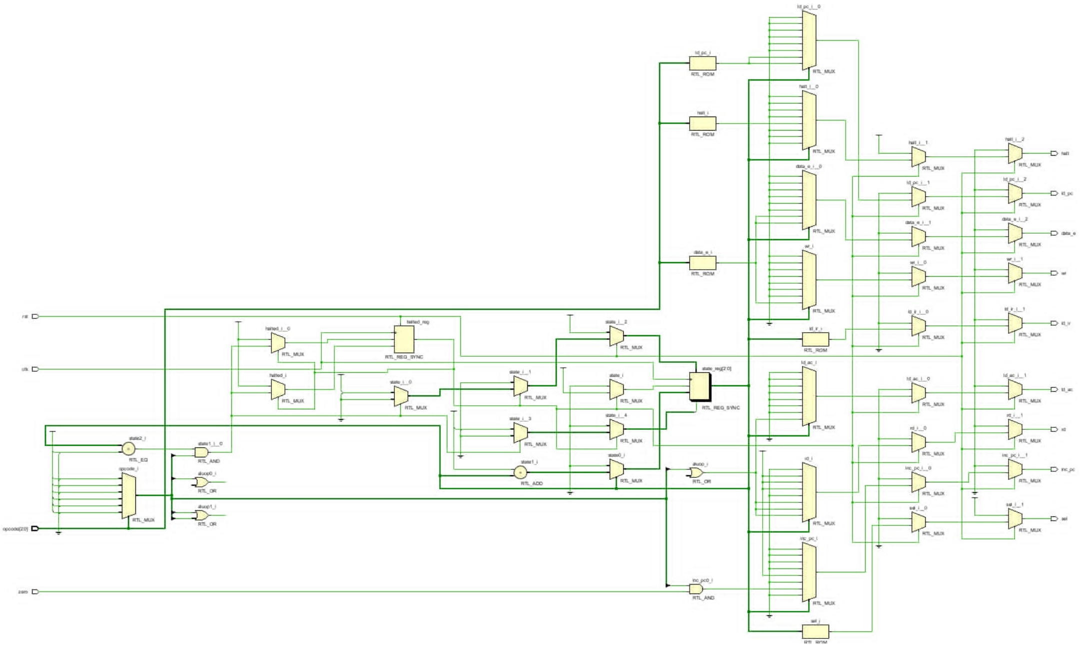
```
                OP_ADDR=3'd4,
                OP_FETCH=3'd5,
                ALU_OP=3'd6,
                STORE=3'd7;

// Opcodes
localparam HLT = 3'b000,
           SKZ = 3'b001,
           ADD = 3'b010,
           AND = 3'b011,
           XOR = 3'b100,
           LDA = 3'b101,
           STO = 3'b110,
           JMP = 3'b111;

reg [2:0] state;
reg halted;

wire aluop;
assign aluop = (opcode == ADD) || (opcode == AND) || (opcode == XOR) ||
(opcode == LDA);

// State register
always @(posedge clk) begin
  if (rst) begin
    state  <= INST_ADDR;
    halted <= 1'b0;
  end else if (halted) begin
    state  <= state;
    halted <= 1'b1;
  end else begin
    if ((state == OP_ADDR) && (opcode == HLT)) begin
      state  <= OP_ADDR;
      halted <= 1'b1;
    end else begin
      state <= (state == STORE) ? INST_ADDR : state + 3'd1;
```

```
      end
    end
  end

  // Output logic (combinational)
  always @(*) begin
    // Default values
    sel    = 1'b0;
    rd     = 1'b0;
    ld_ir  = 1'b0;
    halt   = 1'b0;
    inc_pc = 1'b0;
    ld_ac  = 1'b0;
    ld_pc  = 1'b0;
    wr     = 1'b0;
    data_e = 1'b0;

    if (rst) begin
      sel = 1'b1;
    end else if (halted) begin
      halt = 1'b1;
    end else begin
      case (state)

        INST_ADDR: begin
          sel = 1'b1;
        end

        INST_FETCH: begin
          sel = 1'b1;
          rd  = 1'b1;
        end

        INST_LOAD: begin
          sel   = 1'b1;
          rd    = 1'b1;
```

```
        ld_ir = 1'b1;
      end

      IDLE: begin
        sel   = 1'b1;
        rd    = 1'b1;
        ld_ir = 1'b1;
      end

      OP_ADDR: begin
        inc_pc = 1'b1;

        if (opcode == HLT) begin
          halt = 1'b1;
        end
      end

      OP_FETCH: begin
        rd = aluop;
      end

      ALU_OP: begin
        rd = aluop;

        if (opcode == SKZ && zero) begin
          inc_pc = 1'b1;
        end

        if (opcode == JMP) begin
          ld_pc = 1'b1;
        end

        if (opcode == STO) begin
          data_e = 1'b1;
        end
      end
```

```

        STORE: begin
          rd    = aluop;
          ld_ac = aluop;

          if (opcode == JMP) begin
            ld_pc = 1'b1;
          end

          if (opcode == STO) begin
            wr     = 1'b1;
            data_e = 1'b1;
          end
        end

        default: begin
          sel    = 1'b0;
          rd     = 1'b0;
          ld_ir  = 1'b0;
          halt   = 1'b0;
          inc_pc = 1'b0;
          ld_ac  = 1'b0;
          ld_pc  = 1'b0;
          wr     = 1'b0;
          data_e = 1'b0;
        end

      endcase
    end
  end

endmodule
```

RTL Schematic:



3.6 Instruction Register (IR)

Implementation Logic & Steps:

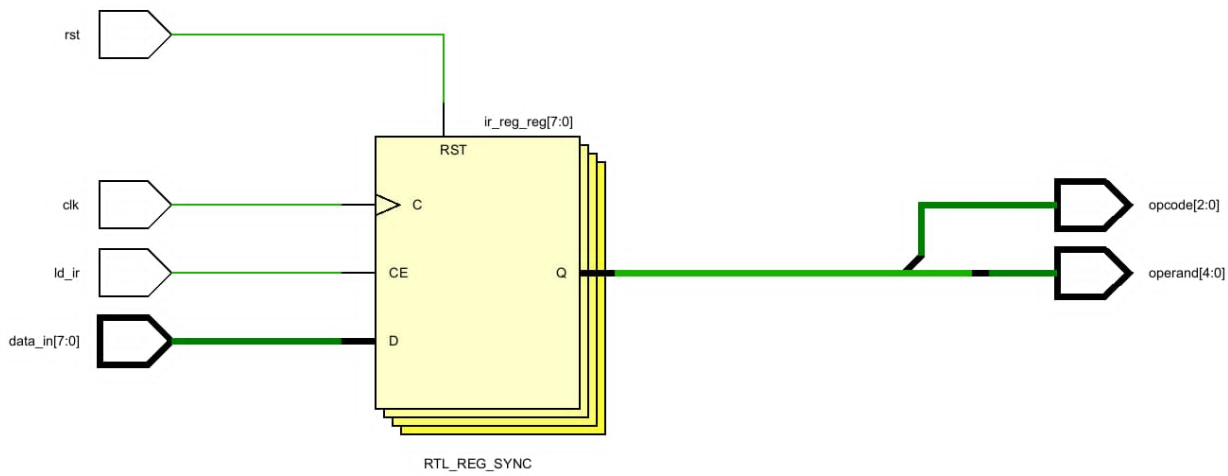
- Declared an internal register to store the instruction data.
- Implemented a sequential logic block triggered by the positive edge of the clock.
- Implemented a synchronous active-high reset to clear the register.
- Implemented logic to load `data_in` into the register when `ld_ir` is active.
- Implemented combinational logic to extract the 3-bit opcode from the stored instruction.
- Implemented combinational logic to extract the 5-bit operand from the stored instruction.

Source Code: `IR.v`:

```
module instruction_register (
    input        clk,
    input        rst,
    input        ld_ir,
    input  [7:0] data_in,
    output [2:0] opcode,
    output [4:0] operand
```

```
);

  reg [7:0] ir_reg;

  always @(posedge clk) begin
    if (rst) begin
      ir_reg <= 8'b00000000;
    end else if (ld_ir) begin
      ir_reg <= data_in[7:0];
    end
  end

  assign opcode  = ir_reg[7:5];

  assign operand = ir_reg[4:0];

endmodule
```

RTL Schematic:



3.7 Accumulator Register (AC)

Implementation Logic & Steps:

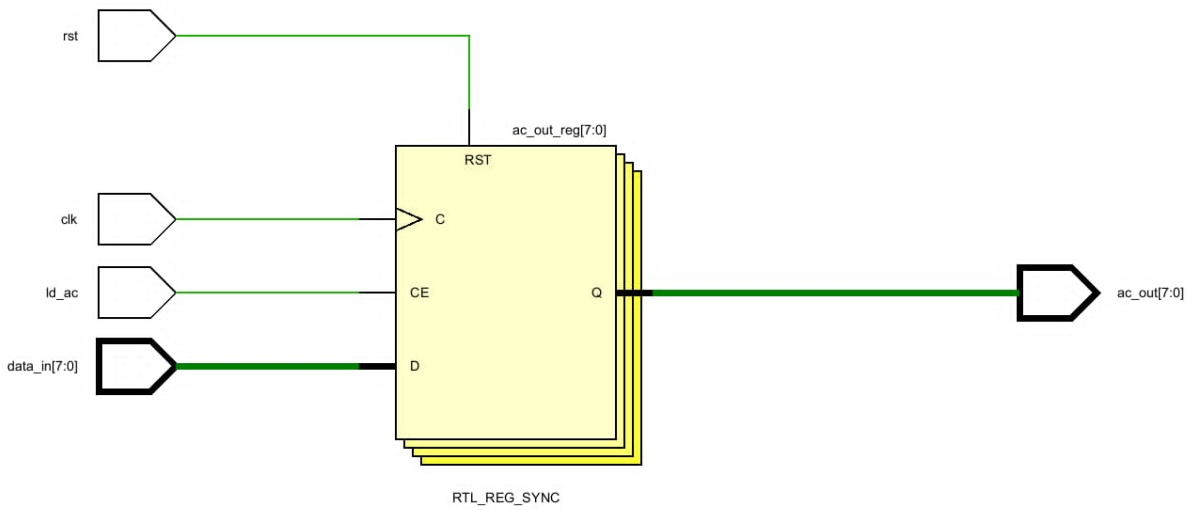
- Implemented a sequential logic block triggered by the positive edge of the clock.

- Implemented a synchronous active-high reset to clear the accumulator.
- Implemented logic to load new data into the accumulator when the load signal is active.

Source Code: AC.v:

```
module accumulator (
    input           clk,
    input           rst,
    input     [7:0] data_in,
    input           ld_ac,
    output reg [7:0] ac_out
);

  always @(posedge clk) begin
    if (rst) ac_out <= 8'b00000000;
    else if (ld_ac) ac_out <= data_in;
    else ac_out <= ac_out;
  end

endmodule
```

RTL Schematic:



3.8 Memory

Implementation Logic & Steps:

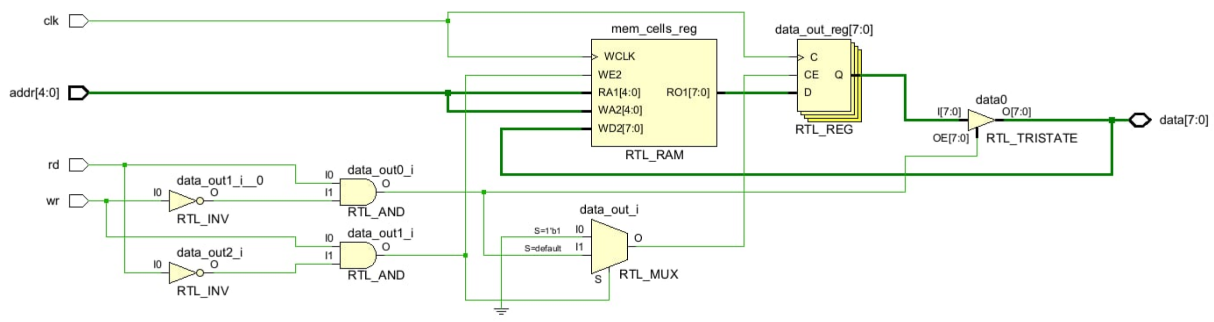
- Declared a 2D array of registers to represent memory cells (32 cells of 8 bits).
- Declared a register to hold the output data.
- Implemented a sequential logic block triggered by the positive edge of the clock for writing data.
- Ensured writing only occurs when `wr` is active and `rd` is inactive.
- Implemented a sequential logic block triggered by the positive edge of the clock for reading data.
- Ensured reading only occurs when `rd` is active and `wr` is inactive.
- Implemented tri-state buffer logic for the bidirectional data bus.

Source Code: `Memory.v`:

```
module Memory (
    input       clk,
    input       rd,
    input       wr,
    input [4:0] addr,
    inout [7:0] data
);

  reg [7:0] mem_cells[31:0];
  reg [7:0] data_out;

  always @(posedge clk) begin
    if (wr && !rd) begin
      mem_cells[addr] <= data;
    end else if (rd && !wr) begin
      data_out <= mem_cells[addr];
    end
  end

  assign data = (rd && !wr) ? data_out : 8'hZZ;
```

```

endmodule
```

RTL Schematic:



3.9 Top-level Integration

Implementation Logic & Steps:

- Declared internal wires for connecting all the CPU submodules.
- Instantiated the Program Counter (PC) module and connected its ports.
- Instantiated the Address Multiplexer (MUX) module and connected its ports.
- Instantiated the ALU module and connected its ports.
- Instantiated the Controller module and connected its ports.
- Instantiated the Instruction Register (IR) module and connected its ports.
- Instantiated the Accumulator (AC) module and connected its ports.
- Instantiated the Memory module and connected its ports.
- Implemented tri-state buffer logic for the data bus to handle writing from the Accumulator to Memory.

Source Code: `CPU.v`:

```
module CPU (
    input  clk,
    input  rst,
    output halt
```

```
);

  // Control signals
  wire sel;
  wire rd;
  wire ld_ir;
  wire inc_pc;
  wire ld_ac;
  wire ld_pc;
  wire wr;
  wire data_e;

  // Datapath signals
  wire [4:0] pc_out;
  wire [4:0] operand;
  wire [4:0] mem_addr;

  wire [2:0] opcode;

  wire [7:0] data_bus;
  wire [7:0] ac_out;
  wire [7:0] alu_out;

  wire zero;

  assign data_bus = (data_e && !rd) ? ac_out : 8'hZZ;

  PC u_pc (
      .clk    (clk),
      .rst    (rst),
      .ld_pc  (ld_pc),
      .inc_pc (inc_pc),
      .data_in(operand),
      .pc_out (pc_out)
  );

```

```
    address_mux #(
        .WIDTH(5)
    ) u_address_mux (
        .pc_addr (pc_out),
        .ir_addr (operand),
        .sel     (sel),
        .addr_out(mem_addr)
    );

    Memory u_memory (
        .clk (clk),
        .rd  (rd),
        .wr  (wr),
        .addr(mem_addr),
        .data(data_bus)
    );

    instruction_register u_ir (
        .clk    (clk),
        .rst    (rst),
        .ld_ir  (ld_ir),
        .data_in(data_bus),
        .opcode (opcode),
        .operand(operand)
    );

    accumulator u_ac (
        .clk    (clk),
        .rst    (rst),
        .data_in(alu_out),
        .ld_ac  (ld_ac),
        .ac_out (ac_out)
    );

    ALU u_alu (
        .opcode (opcode),
```

```
        .inA    (ac_out),
        .inB    (data_bus),
        .alu_out(alu_out),
        .zero   (zero)
    );

    controller u_controller (
        .clk   (clk),
        .rst   (rst),
        .opcode(opcode),
        .zero  (zero),
        .sel   (sel),
        .rd    (rd),
        .ld_ir (ld_ir),
        .halt  (halt),
        .inc_pc(inc_pc),
        .ld_ac (ld_ac),
        .ld_pc (ld_pc),
        .wr    (wr),
        .data_e(data_e)
    );

endmodule
```

RTL Schematic:

4 Testing Simulation and Validation

4.1 Program Counter (PC)

Testbench code: test_001/PC_tb.v:

```
`timescale 1ns / 1ps

// ============================================================
// PC_tb.v - Testbench for Program Counter (PC)
// PC is 5-bit according to the assignment: 32 memory addresses.
```

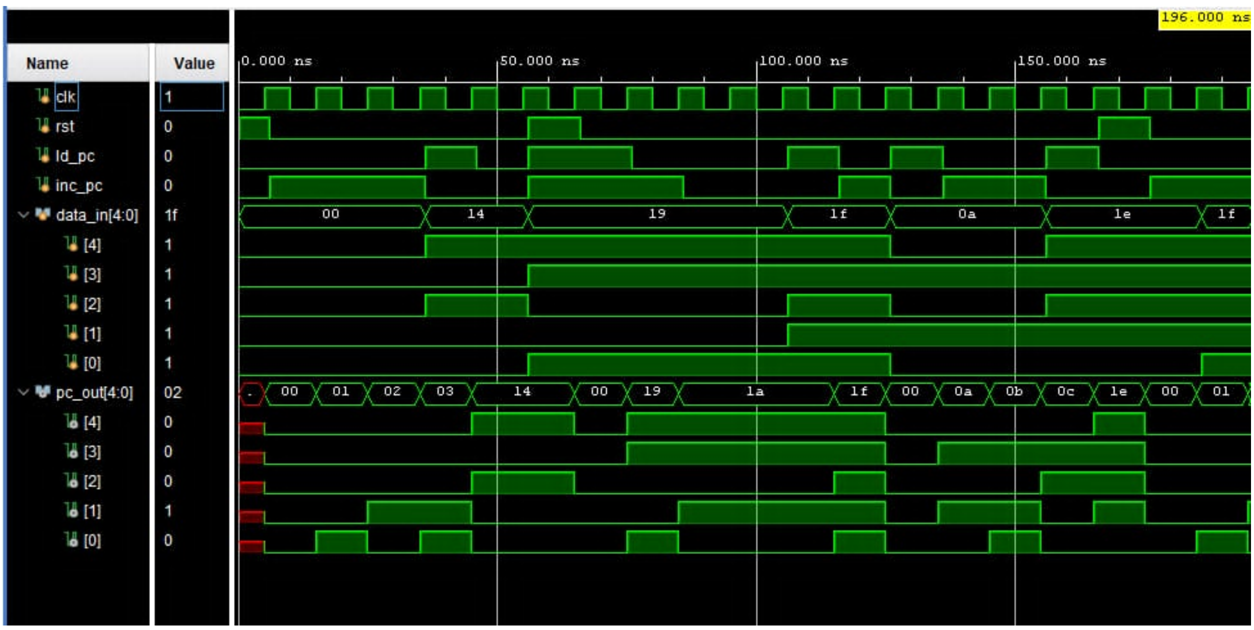
```
// Covers:
//   TC1: Synchronous reset
//   TC2: Increment
//   TC3: Load
//   TC4: Priority rst > ld_pc > inc_pc
//   TC5: Hold
//   TC6: 5-bit wrap-around
//   TC7: Jump & increment
//   TC8: Reset from non-zero value
//   TC9: Data noise immunity
// ============================================================
module PC_tb;

  reg         clk;
  reg         rst;
  reg         ld_pc;
  reg         inc_pc;
  reg   [4:0] data_in;
  wire  [4:0] pc_out;

  PC uut (
      .clk    (clk),
      .rst    (rst),
      .ld_pc  (ld_pc),
      .inc_pc (inc_pc),
      .data_in(data_in),
      .pc_out (pc_out)
  );

  initial clk = 0;
  always #5 clk = ~clk;

  task tick;
    input [63:0] cycle;
    begin
      @(posedge clk);
```

```
      #1;
      $display("cycle=%0d | rst=%b | ld_pc=%b | inc_pc=%b | data_in=%0d |
      pc_out=%0d", cycle, rst,
               ld_pc, inc_pc, data_in, pc_out);
    end
  endtask

  initial begin
    rst = 1;
    ld_pc = 0;
    inc_pc = 0;
    data_in = 5'd0;

    $display("--- TC1: Reset ---");
    tick(1);  // pc_out = 0
    rst = 0;

    $display("--- TC2: Increment ---");
    inc_pc = 1;
    tick(2);  // pc_out = 1
    tick(3);  // pc_out = 2
    tick(4);  // pc_out = 3
    inc_pc = 0;

    $display("--- TC3: Load ---");
    data_in = 5'd20;
    ld_pc   = 1;
    tick(5);  // pc_out = 20
    ld_pc = 0;
    tick(6);  // hold 20

    $display("--- TC4: Priority rst > ld_pc > inc_pc ---");
    data_in = 5'd25;
    rst = 1;
    ld_pc = 1;
    inc_pc = 1;
```

```
    tick(7);  // reset dominates -> 0
    rst = 0;
    tick(8);  // ld_pc dominates -> 25
    ld_pc = 0;
    tick(9);  // inc_pc -> 26
    inc_pc = 0;

    $display("--- TC5: Hold State ---");
    tick(10);  // hold 26
    tick(11);  // hold 26

    $display("--- TC6: 5-bit Wrap-around ---");
    data_in = 5'd31;
    ld_pc   = 1;
    tick(12);  // pc_out = 31
    ld_pc  = 0;
    inc_pc = 1;
    tick(13);  // pc_out wraps to 0
    inc_pc = 0;

    $display("--- TC7: Jump & Increment ---");
    data_in = 5'd10;
    ld_pc   = 1;
    tick(14);  // pc_out = 10
    ld_pc  = 0;
    inc_pc = 1;
    tick(15);  // 11
    tick(16);  // 12
    inc_pc = 0;

    $display("--- TC8: Reset from Non-Zero Value ---");
    data_in = 5'd30;
    ld_pc   = 1;
    tick(17);  // pc_out = 30
    ld_pc = 0;
    rst   = 1;
```

```
    tick(18);  // pc_out = 0
    rst = 0;

    $display("--- TC9: Data Noise Immunity ---");
    inc_pc = 1;
    tick(19);  // pc_out = 1
    data_in = 5'd31;
    tick(20);  // pc_out = 2, not affected by data_in
    inc_pc = 0;

    $display("--- DONE ---");
    $finish;
  end

endmodule
```

Waveform:



4.2 Address Multiplexer (Address MUX)

Testbench code: `test_002/AM_tb.v`:

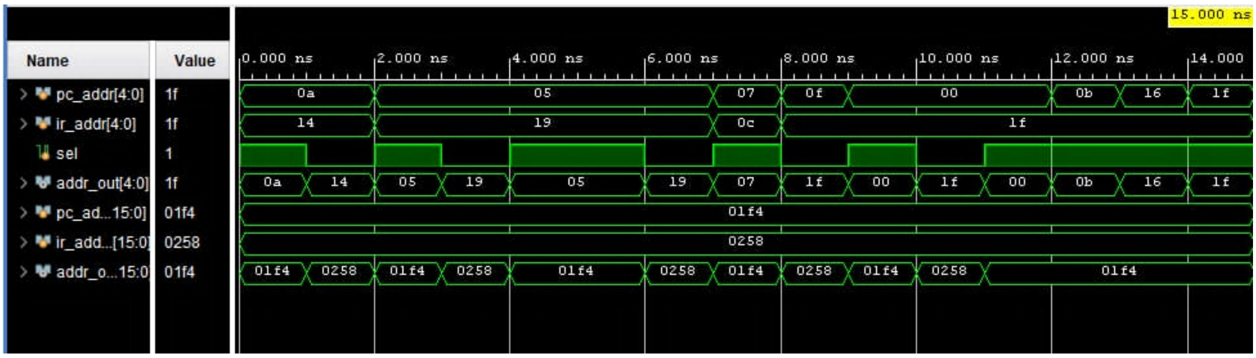
```
`timescale 1ns / 1ps

// ============================================================
// AM_tb.v - Testbench for Address Mux
// Default WIDTH is 5 because CPU memory has 32 addresses.
// Covers:
//    TC1: Select pc_addr
//    TC2: Select ir_addr
//    TC3: Toggle sel
//    TC4: Parameter override
//    TC5: Stability
//    TC6: Edge case addresses
//    TC7: Async response
// ============================================================
module AM_tb;

  reg  [4:0] pc_addr;
  reg  [4:0] ir_addr;
  reg        sel;
  wire [4:0] addr_out;

  address_mux uut (
      .pc_addr (pc_addr),
      .ir_addr (ir_addr),
      .sel     (sel),
      .addr_out(addr_out)
  );

  reg  [15:0] pc_addr_16;
  reg  [15:0] ir_addr_16;
  wire [15:0] addr_out_16;

  address_mux #(
      .WIDTH(16)
  ) uut_16 (
      .pc_addr (pc_addr_16),
```

```
      .ir_addr (ir_addr_16),
      .sel     (sel),
      .addr_out(addr_out_16)
  );

  task display_state;
    input [8*5:1] tc_name;
    begin
      #1;
      $display("%s | sel=%b | pc_addr=%0d | ir_addr=%0d | addr_out=%0d",
      tc_name, sel, pc_addr,
               ir_addr, addr_out);
    end
  endtask

  initial begin
    pc_addr = 5'd10;
    ir_addr = 5'd20;
    sel = 0;

    pc_addr_16 = 16'd500;
    ir_addr_16 = 16'd600;

    $display("--- TC1: Select pc_addr ---");
    sel = 1;
    display_state("TC1  ");

    $display("--- TC2: Select ir_addr ---");
    sel = 0;
    display_state("TC2  ");

    $display("--- TC3: Toggle sel ---");
    pc_addr = 5'd5;
    ir_addr = 5'd25;
    sel = 1;
    display_state("TC3.1");
```

```
    sel = 0;
    display_state("TC3.2");
    sel = 1;
    display_state("TC3.3");

    $display("--- TC4: Parameter Override (WIDTH=16) ---");
    sel = 1;
    #1;
    $display("TC4.1 | sel=%b | pc_addr_16=%0d | ir_addr_16=%0d |
    addr_out_16=%0d", sel, pc_addr_16,
             ir_addr_16, addr_out_16);
    sel = 0;
    #1;
    $display("TC4.2 | sel=%b | pc_addr_16=%0d | ir_addr_16=%0d |
    addr_out_16=%0d", sel, pc_addr_16,
             ir_addr_16, addr_out_16);

    $display("--- TC5: Stability Check ---");
    sel = 1;
    pc_addr = 5'd7;
    ir_addr = 5'd12;
    display_state("TC5.1");
    sel = 0;
    pc_addr = 5'd15;
    ir_addr = 5'd31;
    display_state("TC5.2");

    $display("--- TC6: Edge Case Addresses ---");
    sel = 1;
    pc_addr = 5'd0;
    ir_addr = 5'd31;
    display_state("TC6.1");
    sel = 0;
    display_state("TC6.2");

    $display("--- TC7: Async Response ---");
```

```
    sel = 1;
    #1;
    pc_addr = 5'd11;
    display_state("TC7.1");
    pc_addr = 5'd22;
    display_state("TC7.2");
    pc_addr = 5'd31;
    display_state("TC7.3");

    $display("--- DONE ---");
    $finish;
  end

endmodule
```

Waveform:



4.3 Arithmetic Logic Unit (ALU)

Testbench code: `test_003/ALU_tb.v`:

```
`timescale 1ns / 1ps

// ============================================================
// ALU_tb.v - Testbench for Arithmetic Logic Unit (ALU)
// Covers 7 scenarios:
//   TC1: Zero Status Flag
//   TC2: Arithmetic & Logic
//   TC3: Transfer Operations
```

```
//   TC4: Combinational Property
//   TC5: Boundary Cases
//   TC6: Complex Bitwise Logic
//   TC7: Zero Flag Independence
// ==============================================================
module ALU_tb;

  reg  [7:0] inA;
  reg  [7:0] inB;
  reg  [2:0] opcode;

  wire [7:0] alu_out;
  wire       zero;

  // DUT
  ALU uut (
      .inA    (inA),
      .inB    (inB),
      .opcode (opcode),
      .alu_out(alu_out),
      .zero   (zero)
  );

  task display_state;
    input [8*6:1] tc_name;
    begin
      #1;
      $display("%s | opcode=%b | inA=%0d | inB=%0d | zero=%b | alu_out=%0d",
      tc_name, opcode, inA,
                inB, zero, alu_out);
    end
  endtask

  initial begin

    // Initialize
```

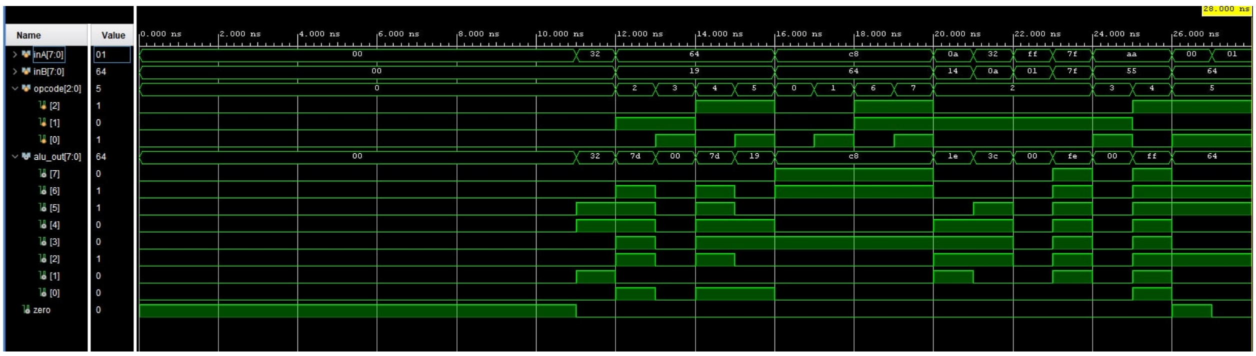
```
    inA = 0;
    inB = 0;
    opcode = 3'b000;
    #10;

    // TC1: Zero Status Flag
    $display("\n--- TC1: Zero Status Flag ---");

    inA = 8'd0;
    display_state("TC1.1");  // zero = 1

    inA = 8'd50;
    display_state("TC1.2");  // zero = 0

    // TC2: Arithmetic & Logic Operations
    $display("\n--- TC2: Arithmetic & Logic ---");

    inA = 8'd100;
    inB = 8'd25;

    // ADD
    opcode = 3'b010;
    display_state("ADD");  // 125

    // AND
    opcode = 3'b011;
    display_state("AND");  // 0

    // XOR
    opcode = 3'b100;
    display_state("XOR");  // 125

    // LDA
    opcode = 3'b101;
    display_state("LDA");  // 25

```

```

    // TC3: Transfer Instructions
    $display("\n--- TC3: HLT / SKZ / STO / JMP ---");

    inA = 8'd200;
    inB = 8'd100;

    // HLT
    opcode = 3'b000;
    display_state("HLT");  // 200

    // SKZ
    opcode = 3'b001;
    display_state("SKZ");  // 200

    // STO
    opcode = 3'b110;
    display_state("STO");  // 200

    // JMP
    opcode = 3'b111;
    display_state("JMP");  // 200


    // TC4: Combinational Property
    $display("\n--- TC4: Combinational Property ---");

    opcode = 3'b010;  // ADD

    inA = 8'd10;
    inB = 8'd20;
    #1;

    $display("TC4.1 | alu_out=%0d (Expected 30)", alu_out);

    // Change inputs immediately
```

```
    inA = 8'd50;
    inB = 8'd10;
    #1;

    $display("TC4.2 | alu_out=%0d (Expected 60)", alu_out);

    // TC5: Boundary Cases
    $display("\n--- TC5: Boundary Cases ---");

    opcode = 3'b010;  // ADD

    inA = 8'hFF;
    inB = 8'h01;
    display_state("OVF1");  // Expect 0

    inA = 8'h7F;
    inB = 8'h7F;
    display_state("OVF2");  // Expect 254

    // TC6: Complex Bitwise Logic
    $display("\n--- TC6: Complex Bitwise Logic ---");

    inA = 8'hAA;  // 10101010
    inB = 8'h55;  // 01010101

    opcode = 3'b011;
    display_state("AND_C");  // 0

    opcode = 3'b100;
    display_state("XOR_C");  // 255


    // TC7: Zero Flag Independence
    $display("\n--- TC7: Zero Flag Independence ---");

    opcode = 3'b101;  // LDA
```

```

    inA = 8'd0;
    inB = 8'd100;
    #1;
    $display("TC7.1 | zero=%b | alu_out=%0d", zero, alu_out);

    inA = 8'd1;
    inB = 8'd100;
    #1;
    $display("TC7.2 | zero=%b | alu_out=%0d", zero, alu_out);

    $display("\n--- ALL TESTS FINISHED ---");
    $finish;

  end

endmodule
```

Waveform:



4.4 Controller Unit

Testbench code: test_004/Controller_tb.v:

```
`timescale 1ns / 1ps

// ============================================================
// Controller_tb.v - Testbench for Controller
// Covers:
```

```
//   TC1: Reset
//   TC2: Fetch phase
//   TC3: ADD execution
//   TC4: STO execution
//   TC5: JMP execution
//   TC6: HLT stable halt
//   TC7: SKZ with zero=0 and zero=1
// ============================================================
module Controller_tb;

  reg       clk;
  reg       rst;
  reg [2:0] opcode;
  reg       zero;

  wire sel, rd, ld_ir, halt, inc_pc, ld_ac, ld_pc, wr, data_e;

  controller uut (
      .clk   (clk),
      .rst   (rst),
      .opcode(opcode),
      .zero  (zero),
      .sel   (sel),
      .rd    (rd),
      .ld_ir (ld_ir),
      .halt  (halt),
      .inc_pc(inc_pc),
      .ld_ac (ld_ac),
      .ld_pc (ld_pc),
      .wr    (wr),
      .data_e(data_e)
  );

  initial clk = 0;
  always #5 clk = ~clk;

```

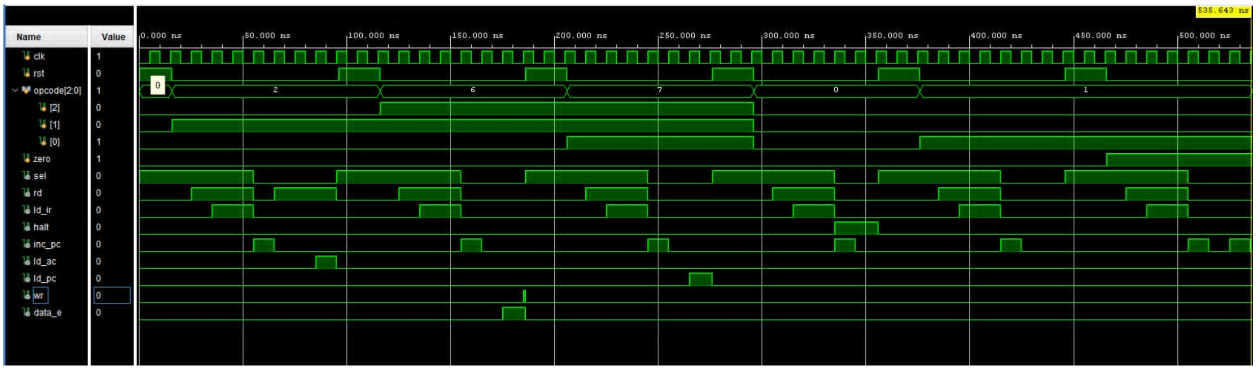
```
task tick;
  begin
    @(posedge clk);
    #1;
  end
endtask

task reset_controller;
  begin
    rst = 1;
    repeat (2) @(posedge clk);
    #1;
    rst = 0;
  end
endtask

task display_state;
  input [8*14:1] state_name;
  begin
    $display(
        "%s | state=%3b opcode=%3b zero=%b | sel=%b rd=%b ld_ir=%b halt=%b
        inc_pc=%b ld_ac=%b ld_pc=%b wr=%b data_e=%b",
        state_name, uut.state, opcode, zero, sel, rd, ld_ir, halt, inc_pc,
        ld_ac, ld_pc, wr,
        data_e);
  end
endtask

initial begin
  opcode = 3'b000;
  zero   = 1'b0;
  rst    = 1'b1;

  reset_controller();

  $display("--- TC1: System Reset / INST_ADDR ---");
```

```
    display_state("0:INST_ADDR   ");

    $display("--- TC2: Fetch Phase ---");
    opcode = 3'b010;  // ADD
    display_state("0:INST_ADDR   ");
    tick();
    display_state("1:INST_FETCH  ");
    tick();
    display_state("2:INST_LOAD   ");
    tick();
    display_state("3:IDLE        ");

    $display("--- TC3: Execution Phase (ADD) ---");
    tick();
    display_state("4:OP_ADDR     ");
    tick();
    display_state("5:OP_FETCH    ");
    tick();
    display_state("6:ALU_OP      ");
    tick();
    display_state("7:STORE       ");
    tick();
    display_state("0:INST_ADDR   ");

    $display("--- TC4: Execution Phase (STO) ---");
    reset_controller();
    opcode = 3'b110;  // STO
    tick();
    tick();
    tick();
    tick();
    display_state("4:OP_ADDR     ");
    tick();
    display_state("5:OP_FETCH    ");
    tick();
    display_state("6:ALU_OP      ");
```

```
    tick();
    display_state("7:STORE       ");

    $display("--- TC5: Execution Phase (JMP) ---");
    reset_controller();
    opcode = 3'b111;  // JMP
    tick();
    tick();
    tick();
    tick();
    display_state("4:OP_ADDR     ");
    tick();
    display_state("5:OP_FETCH    ");
    tick();
    display_state("6:ALU_OP      ");
    tick();
    display_state("7:STORE       ");

    $display("--- TC6: Execution Phase (HLT, stable halt) ---");
    reset_controller();
    opcode = 3'b000;  // HLT
    tick();
    tick();
    tick();
    tick();
    display_state("4:OP_ADDR/HLT ");
    tick();
    display_state("HOLD_HALT_1   ");
    tick();
    display_state("HOLD_HALT_2   ");

    $display("--- TC7.1: SKZ zero=0, no skip ---");
    reset_controller();
    opcode = 3'b001;  // SKZ
    zero   = 1'b0;
    tick();
```

```
    tick();
    tick();
    tick();
    display_state("4:OP_ADDR     ");
    tick();
    display_state("5:OP_FETCH    ");
    tick();
    display_state("6:ALU_OP      ");  // inc_pc should be 0
    tick();
    display_state("7:STORE       ");

    $display("--- TC7.2: SKZ zero=1, skip next instruction ---");
    reset_controller();
    opcode = 3'b001;  // SKZ
    zero   = 1'b1;
    tick();
    tick();
    tick();
    tick();
    display_state("4:OP_ADDR     ");
    tick();
    display_state("5:OP_FETCH    ");
    tick();
    display_state("6:ALU_OP      ");  // inc_pc should be 1
    tick();
    display_state("7:STORE       ");

    $display("--- DONE ---");
    $finish;
  end

endmodule
```

Waveform:



4.5 Instruction Register (IR)

Testbench code: `test_005/IR_tb.v`:

```
`timescale 1ns / 1ps

// ============================================================
// IR_tb.v - Testbench for Instruction Register (IR)
// Covers 7 scenarios described in the project spec:
//    TC1: System Reset
//    TC2: Load Instruction
//    TC3: Hold Value
//    TC4: Extract Opcode and Operand
//    TC5: Reset Priority
//    TC6: Bus Noise Immunity
//    TC7: Instruction Decoding
// ============================================================

module IR_tb;
  // - Signals ---------------------
  reg clk;
  reg rst;
  reg ld_ir;
  reg [7:0] data_in;

  wire [2:0] opcode;
  wire [4:0] operand;
```

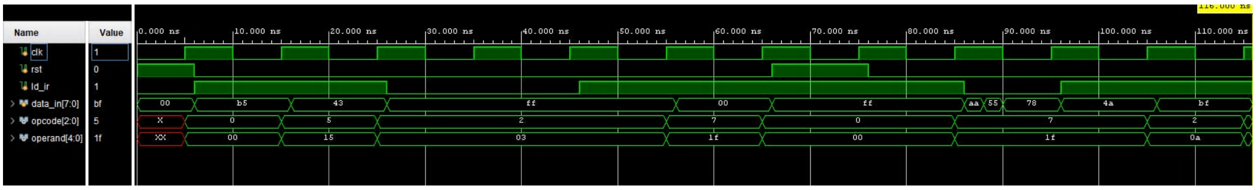
```

// - DUT instantiation ----------------
instruction_register uut (
    .clk    (clk),
    .rst    (rst),
    .ld_ir  (ld_ir),
    .data_in(data_in),
    .opcode (opcode),
    .operand(operand)
);

// - Clock: 10 ns period ---------------
initial begin
  clk = 0;
  forever #5 clk = ~clk;
end

// - Task: tick one clock and display state ------
task display_state;
  input [8*5:1] tc_name;
  begin
    @(posedge clk);
    #1;  // Wait after posedge for data to update
    $display("%s | rst=%b | ld_ir=%b | data_in=0x%02X | opcode=%b |
    operand=%b", tc_name, rst,
             ld_ir, data_in, opcode, operand);
  end
endtask

initial begin
  // Init
  rst = 1;
  ld_ir = 0;
  data_in = 8'd0;
  #1;
  // - TC1: System Reset --------------
```

```
    $display("--- TC1: System Reset ---");
    display_state("TC1.1");  // Expect opcode=0, operand=0

    rst = 0;  // De-assert reset

    // - TC2: Load Instruction ------------
    $display("--- TC2: Load Instruction ---");
    ld_ir   = 1;
    // Instruction 1: Opcode = 101 (LDA), Operand = 10101
    // Combined: [7:5] = 101, [4:0] = 10101 -> 1011_0101 = 0xB5
    data_in = 8'hB5;
    display_state("TC2.1");  // Expect: opcode=101, operand=10101

    // Instruction 2: Opcode = 010 (ADD), Operand = 00011
    // Combined: [7:5] = 010, [4:0] = 00011 -> 0100_0011 = 0x43
    data_in = 8'h43;
    display_state("TC2.2");  // Expect: opcode=010, operand=00011

    // - TC3: Hold Value ---------------
    $display("--- TC3: Hold Value ---");
    ld_ir   = 0;  // De-assert load instruction signal
    data_in = 8'hFF;  // Change data on bus
    display_state("TC3.1");  // Expect: opcode=010, operand=00011 (holds value)
    display_state("TC3.2");  // Expect: opcode=010, operand=00011

    // - TC4: Extract Opcode and Operand -------
    $display("--- TC4: Extract Opcode and Operand ---");
    // Instruction 3: Opcode = 111 (JMP), Operand = 11111
    // Combined: [7:5] = 111, [4:0] = 11111 -> 1111_1111 = 0xFF
    ld_ir   = 1;
    data_in = 8'hFF;
    display_state("TC4.1");  // Expect: opcode=111, operand=11111

    // Instruction 4: Opcode = 000 (HLT), Operand = 00000
    data_in = 8'h00;
```

```
    display_state("TC4.2");  // Expect: opcode=000, operand=00000

    // - TC5: Reset Priority -------------
    $display("--- TC5: Reset Priority ---");
    ld_ir   = 1;
    data_in = 8'hFF;  // Attempt to load all 1s
    rst     = 1;  // But assert reset simultaneously
    display_state("TC5.1");  // Expect: opcode=000, operand=00000
    rst = 0;
    display_state("TC5.2");  // After reset is de-asserted, FF data is loaded (opcode=111, op=11111)

    // - TC6: Bus Noise Immunity -----------
    // Change data_in continuously when ld_ir = 0
    $display("--- TC6: Bus Noise Immunity ---");
    ld_ir   = 0;
    data_in = 8'hAA;
    #2;
    data_in = 8'h55;
    #2;
    data_in = 8'h78;
    display_state("TC6.1");  // Expect: holds value from TC5.2 (111, 11111)

    // - TC7: Instruction Decoding ----------
    $display("--- TC7: Instruction Decoding ---");
    ld_ir   = 1;
    // Load ADD (010) Operand (01010)
    data_in = 8'h4A;  // 4A = 010_01010
    display_state("TC7.1");  // Expect: opcode=010, operand=01010

    // Load LDA (101) Operand (11111)
    data_in = 8'hBF;  // BF = 101_11111
    display_state("TC7.2");  // Expect: opcode=101, operand=11111

    $display("--- DONE ---");
    $finish;
```

```
  end
endmodule
```

Waveform:



4.6 Accumulator Register (AC)

Testbench code: `test_006/AC_tb.v`:

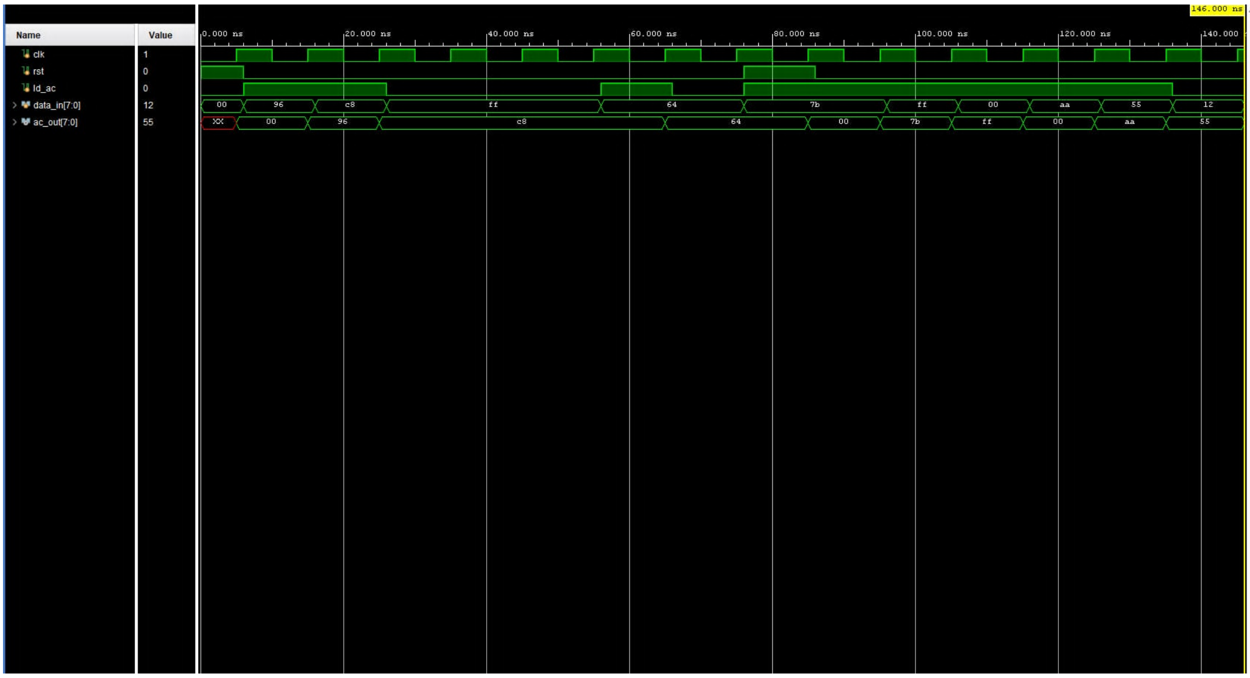
```
`timescale 1ns / 1ps

// ============================================================
// AC_tb.v - Testbench for Accumulator (AC)
// Covers 8 scenarios:
//   TC1: System Reset
//   TC2: Load Data
//   TC3: Data Stability
//   TC4: Feedback to ALU
//   TC5: Interaction with Data Bus
//   TC6: Reset Priority
//   TC7: Boundary Values
//   TC8: Rapid Toggling
// ============================================================
module AC_tb;

  // - Signals --------------------
  reg         clk;
  reg         rst;
  reg         ld_ac;
  reg   [7:0] data_in;
  wire  [7:0] ac_out;

```

```
// - DUT instantiation ----------------
accumulator uut (
    .clk    (clk),
    .rst    (rst),
    .ld_ac  (ld_ac),
    .data_in(data_in),
    .ac_out (ac_out)
);

// - Clock: 10 ns period ---------------
initial begin
  clk = 0;
  forever #5 clk = ~clk;
end

// - Task: tick one clock and display state ------
task display_state;
  input [8*5:1] tc_name;
  begin
    @(posedge clk);
    #1;  // Wait after posedge for data to stabilize
    $display("%s | rst=%b | ld_ac=%b | data_in=%0d | ac_out=%0d", tc_name,
    rst, ld_ac, data_in,
             ac_out);
  end
endtask

initial begin
  // Init
  rst = 1;
  ld_ac = 0;
  data_in = 8'd0;
  #1;

  // - TC1: System Reset --------------
  $display("--- TC1: System Reset ---");
```

```
    display_state("TC1  ");  // Expect: ac_out = 0
    rst = 0;

    // - TC2: Load Data (ALU Result) ---------
    $display("--- TC2: Load Data (ALU Result) ---");
    ld_ac   = 1;
    data_in = 8'd150;  // Assume result from ALU is 150
    display_state("TC2.1");  // Expect: ac_out = 150

    data_in = 8'd200;
    display_state("TC2.2");  // Expect: ac_out = 200

    // - TC3: Data Stability (Idle/Fetch) -------
    $display("--- TC3: Data Stability (Idle/Fetch) ---");
    ld_ac   = 0;
    data_in = 8'd255;  // Change input data
    display_state("TC3.1");  // Expect: ac_out holds 200
    display_state("TC3.2");  // Expect: ac_out holds 200

    // - TC4: Feedback to ALU -------------
    $display("--- TC4: Feedback to ALU ---");
    // This case verifies that ac_out (inA of ALU) is always ready
    display_state("TC4  ");  // Expect: ac_out = 200

    // - TC5: Interaction with Data Bus --------
    $display("--- TC5: Interaction with Data Bus ---");
    // Simulate loading a new value in preparation to write to Memory
    ld_ac   = 1;
    data_in = 8'd100;
    display_state("TC5.1");  // Expect: ac_out = 100
    ld_ac = 0;
    display_state("TC5.2");  // Value holds stable during write cycle

    // - TC6: Reset Priority -------------
    $display("--- TC6: Reset Priority Check ---");
    ld_ac   = 1;
```

```
    data_in = 8'd123;
    rst     = 1;  // Assert Reset while load is active
    display_state("TC6.1");  // Expect: ac_out = 0 (Reset wins)
    rst = 0;
    display_state("TC6.2");  // Expect: ac_out = 123 (Normal load resumes)

    // - TC7: Boundary Values -------------
    $display("--- TC7: Boundary Values ---");
    ld_ac   = 1;
    data_in = 8'hFF;  // All 1s
    display_state("TC7.1");
    data_in = 8'h00;  // All 0s
    display_state("TC7.2");

    // - TC8: Rapid Toggling -------------
    $display("--- TC8: Rapid Toggling ---");
    ld_ac   = 1;
    data_in = 8'hAA;
    display_state("TC8.1");
    data_in = 8'h55;
    display_state("TC8.2");
    ld_ac   = 0;
    data_in = 8'h12;
    display_state("TC8.3");  // Expect: Holds 8'h55

    $display("--- DONE ---");
    $finish;
  end
endmodule
```

Waveform:



4.7 Memory

Testbench code: `test_007/Memory_tb.v`:

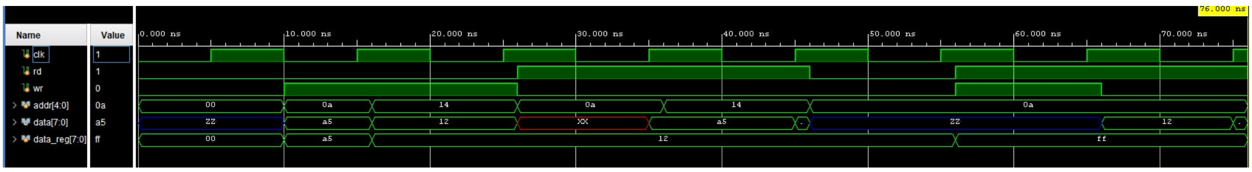
```
`timescale 1ns / 1ps

// ============================================================
// Memory_tb.v - Testbench for Memory
// Covers 5 scenarios described in the project spec:
//    TC1: Write Data
//    TC2: Read Data
//    TC3: High Impedance Check
//    TC4: Conflict Prevention
//    TC5: Data Persistence
// ============================================================
module Memory_tb;

  // - Signals ---------------------
  reg         clk;
  reg         rd;
```

```
  reg        wr;
  reg  [4:0] addr;  // 5-bit address for 32 Memory cells
  wire [7:0] data;  // Bidirectional bus

  // Bus control signals from Testbench
  reg  [7:0] data_reg;

  // Inout control logic: Write only when wr=1 and rd=0
  assign data = (wr && !rd) ? data_reg : 8'hZZ;

  // - DUT instantiation ----------------
  Memory uut (
      .clk (clk),
      .rd  (rd),
      .wr  (wr),
      .addr(addr),
      .data(data)
  );

  // - Clock: 10 ns period ---------------
  initial begin
    clk = 0;
    forever #5 clk = ~clk;
  end

  // - Task: tick one clock and display state ------
  task display_state;
    input [8*5:1] tc_name;
    begin
      @(posedge clk);
      #1;  // Wait after posedge for data to stabilize
      $display("%s | rd=%b | wr=%b | addr=%0d | data=0x%02X", tc_name, rd,
      wr, addr, data);
    end
  endtask

```

```
  initial begin
    // Init
    rd       = 0;
    wr       = 0;
    addr     = 0;
    data_reg = 0;
    #10;

    // - TC1: Write Data ---------------
    $display("--- TC1: Write Data ---");
    wr       = 1;
    rd       = 0;
    addr     = 5'd10;
    data_reg = 8'hA5;
    display_state("TC1.1");  // Write to Memory cell 10

    addr     = 5'd20;
    data_reg = 8'h12;
    display_state("TC1.2");  // Write to Memory cell 20
    wr = 0;

    // - TC2: Read Data ----------------
    $display("--- TC2: Read Data ---");
    wr   = 0;
    rd   = 1;
    addr = 5'd10;
    display_state("TC2.1");  // Expect: A5
    addr = 5'd20;
    display_state("TC2.2");  // Expect: 12

    // - TC3: High Impedance Check ----------
    $display("--- TC3: High Impedance Check ---");
    wr   = 0;
    rd   = 0;  // No read, no write
    addr = 5'd10;
    display_state("TC3  ");  // Expect: data = ZZ
```

```

    // - TC4: Conflict Prevention -----------
    $display("--- TC4: Conflict Prevention ---");
    // Per spec: Simultaneous read and write not allowed
    wr       = 1;
    rd       = 1;
    addr     = 5'd10;
    data_reg = 8'hFF;
    display_state("TC4  ");  // Expect: System must handle safely (usually
    high-Z or priority)

    // - TC5: Data Persistence ------------
    $display("--- TC5: Data Persistence ---");
    wr   = 0;
    rd   = 1;
    addr = 5'd10;
    display_state("TC5  ");  // Confirm data in cell 10 is still A5 after
    conflict

    $display("--- DONE ---");
    $finish;
  end
endmodule
```

Waveform:



4.8 Top-level Integration

Testbench code: `test_008/CPU_tb.v`:

```
`timescale 1ns / 1ps

// ==============================================================
```

```
// CPU_tb.v - Top-Level Testbench for Integration (CPU)
//    - LDA 20
//    - ADD 21
//    - STO 22
//    - HLT
// ============================================================
module CPU_tb;

  // - Signals ---------------------
  reg  clk;
  reg  rst;
  wire halt;

  // - DUT instantiation ----------------
  CPU uut (
      .clk (clk),
      .rst (rst),
      .halt(halt)
  );

  // - Clock: 10 ns period ---------------
  initial clk = 0;
  always #5 clk = ~clk;

  // - Task: display CPU state -------------
  task display_cpu_state;
    begin
      $display("time=%t | PC=%2d | State=%3b | Op=%3b | AC=%0d | Halt=%b",
      $time, uut.u_pc.pc_out,
                uut.u_controller.state, uut.opcode, uut.u_ac.ac_out, halt);
    end
  endtask

  // - Load Program & Control Simulation --------
  initial begin
    // 1. Load specific test program
```

```
    // LDA 20, ADD 21, STO 22, HLT
    uut.u_memory.mem_cells[0] = 8'hB4;  // LDA 20 (Opcode 101, Operand 10100)
    uut.u_memory.mem_cells[1] = 8'h55;  // ADD 21 (Opcode 010, Operand 10101)
    uut.u_memory.mem_cells[2] = 8'hD6;  // STO 22 (Opcode 110, Operand 10110)
    uut.u_memory.mem_cells[3] = 8'h00;  // HLT    (Opcode 000, Operand 00000)

    // 2. Load input data
    uut.u_memory.mem_cells[20] = 8'd5;
    uut.u_memory.mem_cells[21] = 8'd3;
    uut.u_memory.mem_cells[22] = 8'd0;  // Location to store result

    // 3. Reset system
    rst = 1;
    repeat (2) @(posedge clk);
    #1;
    rst = 0;

    $display("--- CPU INTEGRATION TEST STARTED ---");
  end

  // Monitor state at each positive clock edge
  always @(posedge clk) begin
    if (!rst) display_cpu_state();
  end

  // Final result check after HLT
  initial begin
    wait (halt === 1'b1);
    repeat (2) @(posedge clk);

    $display("--- FINAL RESULT CHECK ---");
    $display("Mem[22] (Expected 8): %0d", uut.u_memory.mem_cells[22]);
```

```

    if (uut.u_memory.mem_cells[22] == 8) $display("--- TEST STATUS: PASS
    ---");
    else $display("--- TEST STATUS: FAIL ---");

    $finish;
  end

  // Safety Timeout
  initial #10000 $finish;

endmodule
```

test_009/CPU2_tb.v:

```
`timescale 1ns / 1ps

// ==============================================================
// CPU2_tb.v - Top-Level Testbench for Integration (CPU)
// Covers execution of a comprehensive test program evaluating:
//    - LDA, ADD, STO sequence
//    - Conditional SKZ branching
//    - Unconditional JMP branching
//    - Bitwise XOR and AND operations
//    - HLT assertion
// ==============================================================
module CPU2_tb;

  // - Signals ---------------------
  reg  clk;
  reg  rst;
  wire halt;

  // - DUT instantiation ----------------
  CPU uut (
      .clk (clk),
      .rst (rst),
```

```
      .halt(halt)
  );

  // - Clock: 10 ns period ---------------
  initial clk = 0;
  always #5 clk = ~clk;

  // - Task: display CPU state -------------
  // Prints out whenever PC changes or an instruction cycle finishes (8
  phases)
  task display_cpu_state;
    begin
      $display("time=%t | PC=%2d | State=%3b | Op=%3b | AC=%0d | Halt=%b",
      $time, uut.u_pc.pc_out,
               uut.u_controller.state, uut.opcode, uut.u_ac.ac_out, halt);
    end
  endtask

  // - Load Program & Control Simulation --------
  initial begin
    // 1. Load comprehensive test program into Memory
    // Program executes: (5 + 3), XOR 5, AND 3, tests SKZ/JMP
    uut.u_memory.mem_cells[0] = 8'hB4;  // LDA 20 -> AC = 5
    uut.u_memory.mem_cells[1] = 8'h55;  // ADD 21 -> AC = 8
    uut.u_memory.mem_cells[2] = 8'hD6;  // STO 22 -> Mem[22] = 8
    uut.u_memory.mem_cells[3] = 8'h20;  // SKZ    -> AC=8 (!=0) -> No jump
    uut.u_memory.mem_cells[4] = 8'hE6;  // JMP 6  -> PC jumps to 6
    uut.u_memory.mem_cells[5] = 8'h55;  // ADD 21 -> (Skipped due to JMP)
    uut.u_memory.mem_cells[6] = 8'h94;  // XOR 20 -> 8 XOR 5 = 13
    uut.u_memory.mem_cells[7] = 8'h75;  // AND 21 -> 13 AND 3 = 1
    uut.u_memory.mem_cells[8] = 8'hD7;  // STO 23 -> Mem[23] = 1
    uut.u_memory.mem_cells[9] = 8'hB8;  // LDA 24 -> AC = 0 (Data at 24 is
    0)
    uut.u_memory.mem_cells[10] = 8'h20;  // SKZ    -> AC=0 -> Skip next
    instruction
    uut.u_memory.mem_cells[11] = 8'hD9;  // STO 25 -> (Skipped due to SKZ)
```

```
    uut.u_memory.mem_cells[12] = 8'h00;  // HLT    -> Halt

    // 2. Load operand data
    uut.u_memory.mem_cells[20] = 8'd5;
    uut.u_memory.mem_cells[21] = 8'd3;
    uut.u_memory.mem_cells[22] = 8'd0;
    uut.u_memory.mem_cells[23] = 8'd0;
    uut.u_memory.mem_cells[24] = 8'd0;

    // 3. Reset system
    rst = 1;
    repeat (2) @(posedge clk);
    #1;
    rst = 0;

    $display("--- CPU INTEGRATION TEST STARTED ---");
  end

  // Monitor state at each positive clock edge
  always @(posedge clk) begin
    if (!rst) display_cpu_state();
  end

  // Final result check after HLT
  initial begin
    wait (halt === 1'b1);
    repeat (2) @(posedge clk);

    $display("--- FINAL RESULT CHECK ---");
    $display("Mem[22] (Expected 8): %0d", uut.u_memory.mem_cells[22]);
    $display("Mem[23] (Expected 1): %0d", uut.u_memory.mem_cells[23]);

    if (uut.u_memory.mem_cells[22] == 8 && uut.u_memory.mem_cells[23] == 1)
      $display("--- TEST STATUS: PASS ---");
    else $display("--- TEST STATUS: FAIL ---");

```

```
      $finish;
    end

    // Safety Timeout
    initial #10000 $finish;

endmodule
```

Waveform:

5 Conclusion and Future Work

5.1 Conclusion

In this assignment, our group designed and implemented a simple RISC processor using Verilog HDL. The processor follows an accumulator-based architecture and uses an 8-bit instruction format, consisting of a 3-bit opcode and a 5-bit operand field. Based on this format, the CPU supports eight instructions: HLT, SKZ, ADD, AND, XOR, LDA, STO, and JMP. The design also includes a 5-bit program counter and a 32-location memory space, which are consistent with the project specification.

The processor was developed using a modular design approach. Core components, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, ALU, and Controller, were implemented as independent Verilog modules and then integrated into the top-level CPU module. This organization made the datapath and control path clearer, while also making simulation, debugging, and system integration more manageable.

Key part of the design was the Controller, which was implemented as a finite state machine with eight states: `INST_ADDR`, `INST_FETCH`, `INST_LOAD`, `IDLE`, `OP_ADDR`, `OP_FETCH`, `ALU_OP`, and `STORE`. These states coordinate the main phases of instruction execution, including instruction fetch, decoding, operand access, ALU operation, accumulator update, memory write-back, program counter update, and halt control. Through simulation, the processor was verified to execute the supported instructions correctly at both module and system levels.

The main strength of the design is its clear separation between datapath components and control logic. Since each module was tested independently before being connected in the complete CPU, functional errors were easier to identify and correct. In addition, the use of both module-level and CPU-level testbenches helped confirm the correctness of individual blocks as well as the overall instruction execution flow.

However, the current processor still has several limitations. The instruction set is relatively small and does not support more complex arithmetic or comparison operations. The 5-bit address bus also limits the memory space to only 32 locations, which restricts the size of executable programs. Moreover, the controller follows a fixed multi-state execution sequence, meaning that some simple instructions may require more clock cycles than necessary. The project also focuses mainly on functional simulation, while synthesis results, timing delay, resource utilization, and power consumption have not yet been analyzed in detail.

5.2 Future Plan

Based on these limitations, several improvements can be considered for future development. First, the instruction set can be extended to improve the flexibility of the processor. Additional instructions such as SUB, MUL, DIV, shift operations, and comparison instructions would allow the CPU to support more diverse computational tasks. For example, comparison and shift instructions would make the processor more capable of handling conditional operations and bit-level data manipulation.

Second, the memory system can be expanded by increasing the address width. Since the current 5-bit address bus only supports 32 memory locations, a wider address bus would allow the processor to access a larger program and data memory space. This improvement would make it possible to execute longer instruction sequences and store more intermediate data during program execution.

Third, the Controller can be optimized to reduce unnecessary execution cycles. In the current design, instructions follow a fixed sequence of states, even when some operations do not require all stages. Future versions could use instruction-dependent state transitions so that simpler instructions, such as HLT or JMP, complete in fewer cycles. More advanced control techniques, including pipelining and hazard handling, could also be explored to improve instruction throughput.

Fourth, verification should be extended with more comprehensive test programs. Although module-level and CPU-level simulations were performed, additional cases should be tested, such as repeated jump operations, consecutive store instructions, SKZ behavior when the accumulator is zero and non-zero, and boundary memory access at the lowest and highest addresses. These cases would help evaluate the robustness of the processor under less straightforward execution scenarios.

Finally, synthesis and implementation analysis should be performed to evaluate the hardware cost of the design. Resource utilization, critical path delay, maximum operating frequency, and power consumption should be analyzed after synthesis. This would provide a more complete understanding of the processor not only as a functional Verilog model, but also as a hardware design that could be implemented on an FPGA or similar digital platform.

Overall, this assignment helped us understand the fundamental principles of processor design, especially the interaction between datapath modules and control logic. It also provided practical experience in using Verilog HDL, designing a finite state machine controller, integrating multiple hardware modules, and verifying a digital system through simulation.

Member Contribution*

No.	Member	Student ID	Contribution
1	Trần Vinh Quang	2550342	Implemented the Controller module, including the finite state machine and control signal generation. Contributed to the project report.
2	Lê Hiển Vinh	2453422	Implemented the Program Counter and top-level CPU modules. Developed expected output files for testbenches, and performed simulation verification for the system.
3	Nguyễn Đức Thiên Tân	2550345	Implemented the Memory and Instruction Register modules, including memory access logic and instruction decoding. Contributed to the project report.
4	Đặng Ngọc Lan Anh	2550258	Implemented the ALU and Accumulator modules, including arithmetic, logic, zero flag, and register operations. Contributed to the project report.